

Git & GitHub CLI

Everyday `git` plus the `gh` CLI – including juggling a personal and a Salesforce EMU account on one host.

Identity & Config

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --list --show-origin # where each value comes from
```

- Override per-repo (no `--global`) for work vs. personal email.
- Route email by folder automatically – see **Per-Directory Identity**.

Everyday Loop

```
git status -sb # short + branch info
git add -p # stage hunks interactively
git commit -m "message"
git commit --amend # fix the last commit
git switch -c feature # create + switch branch
git switch main
```

- `git switch` / `git restore` are the modern split of `git checkout`.

Branch & Merge

```
git branch -vv # branches + tracking + ahead/behind
git switch -c fix/login-bug
git merge --no-ff feature # keep a merge commit
git branch -d feature # delete merged branch
git push -u origin fix/login-bug
```

Sync With Remotes

```
git remote -v
git remote add origin git@github.com:org/repo.git
git fetch --all --prune # drop deleted remote branches
git pull --rebase # linear history
git push
```

Undo & Rescue

```
git restore file.txt # discard working changes
git restore --staged file.txt # unstage, keep edits
git reset --soft HEAD~1 # undo commit, keep it staged
git reset --hard origin/main # 🗑️ nuke local to match remote
git revert <sha> # safe undo (new commit)
git reflog # find "lost" commits
git stash push -m "wip"; git stash pop
```

Inspect History

```
git log --oneline --graph --decorate --all
git diff # unstaged
git diff --staged # what a commit would capture
git show <sha>
git blame path/to/file
```

Rebase & Rewrite

```
git rebase main # replay branch onto main
git rebase -i HEAD~3 # squash / reword / reorder
git rebase --continue # or --abort
git cherry-pick <sha>
```

- Rewriting shared history needs `git push --force-with-lease`, never plain `--force`.

gh CLI Basics

```
gh repo clone org/repo
gh repo create my-app --private --clone
gh pr create --fill # PR from current branch
gh pr checkout 123
gh pr status
gh issue list --assignee @me
```

- `gh` speaks to whichever account is **active** for the target host.

gh Auth: Many Accounts, One Host

`gh` (since v2.40) stores multiple accounts per host – one **active** at a time. Adding a second account **never overwrites** the first.

```
gh auth status # accounts per host + which is active
gh auth login --hostname github.com # add another account
gh auth switch --hostname github.com --user <login>
gh auth logout --user <login>
```

`gh auth status --hostname github.com` showing only your personal login just means the EMU account isn't added **yet** – not that it replaced anything.

Scenario: Personal + Salesforce EMU

Both live on [github.com](#) – same host. `venkat-viswanathan_sfemu` (the `_sfemu` suffix + `/enterprises/...` URL) is an **Enterprise Managed User** that logs in through Salesforce **SSO/IdP**. Add it alongside `venkatviswa`; don't replace it.

```
# 1. add the EMU account (interactive; needs a browser)
gh auth login --hostname github.com # choose HTTPS

# 2. confirm BOTH now show under github.com
gh auth status

# 3. switch context per task
gh auth switch --hostname github.com --user venkat-viswanathan_sfemu # work
gh auth switch --hostname github.com --user venkatviswa # personal
```

The EMU Login Gotcha

Your browser is already signed in as `venkatviswa`, so the default device-code flow will just re-authorize your **personal** account. To actually land the EMU account, pick one:

- Paste a token (best for EMU):** in an incognito window sign in as `venkat-viswanathan_sfemu` via SSO → **Settings** › **Developer settings** › **PAT**, authorize it for **SSO** → in `gh auth login` choose “Paste an authentication token.”
- Browser flow:** sign out of [github.com](#) (or use incognito), sign in as the EMU user via SSO, then complete the device-code prompt.

“Authorize your device” – Expected

Seeing “Signed in as `venkat-viswanathan_sfemu`” on the *Authorize your device* page is **success**, **not an error** – your browser is on the EMU identity, which is the hard part.

- `gh` can't type into the browser, so it prints an `XXXX-XXXX` code and opens `github.com/login/device`.
- Enter that code → **Authorize** → approve the **SAML SSO / enterprise** prompt too, or the token won't reach enterprise repos.
- Terminal flips to **✓ Authentication complete**; the EMU account is stored next to your personal one.
- “Never use a code sent by someone else” = anti-phishing warning, ignore if the code is yours.

Git Follows the Active Account

```
gh auth setup-git # wires gh as the credential helper
gh auth switch --user <login> # git clone/push/pull now use this account
```

- With `gh` as the credential helper, `git` on [github.com](#) automatically uses whichever account is **currently active** – no separate login.
- EMU accounts see **only** their enterprise; personal can't touch the enterprise. You're switching contexts, not blending them.

Both At Once: Per-Directory Identity

Want work + personal usable side-by-side **without switching**? Route by folder with `includeIf` – commits get the right email automatically.

```
# ~/.gitconfig
[includeIf "gitdir:~/work/"]
path = ~/.gitconfig-work
```

```
# ~/.gitconfig-work
[user]
email = venkat.viswanathan@salesforce.com
```

Both At Once: SSH Host Aliases

```
# ~/.ssh/config
Host github-emu
  HostName github.com
  User git
  IdentityFile ~/.ssh/id_emu

Host github.com
  IdentityFile ~/.ssh/id_personal
```

```
git clone git@github-emu:salesforce-emu/repo.git
```

- Distinct keys let both identities push without switching. **Note:** EMU policy may restrict SSH – HTTPS + `gh` is often the reliable path.

Env Var Overrides

VAR	EFFECT
<code>GH_TOKEN</code>	token for <code>gh</code> (skips stored auth)
<code>GH_HOST</code>	default host for <code>gh</code> commands
<code>GIT_SSH_COMMAND</code>	one-off SSH key: <code>ssh -i ~/.ssh/id_emu</code>

Handy in CI or to force a specific identity for a single command without touching config.

Handy Aliases

```
git config --global alias.st "status -sb"
git config --global alias.lg \
  "log --oneline --graph --decorate --all"
git config --global alias.last "log -1 HEAD"
```

- `gh alias set prc 'pr create --fill'` → `gh prc`.
- `gh` supports shell aliases too: `gh alias set --shell root 'cd $(git rev-parse --show-toplevel)'`.